Universidade Cruzeiro do Sul

Engenharia de Software

Projeto Integrador Transdisciplinar II

Documento de Testes
Loja Virtual de Cupcakes

Aluno:
Fellipe Augusto Gonçalves Magalhães

Novembro de 2026

Sumário

1 Introdução **2**

2 Estratégia de testes **2**

3 Ambiente e como executar **3**

3.1 Ambiente usado 3

3.2 Executar os testes de back-end 3

3.3 Executar os testes de interface 3

4 Resultados **4**

4.1 Back-end e banco de dados 4

4.2 Interface 5

5 Defeitos encontrados e corrigidos **5**

6 Rastreabilidade **5**

6.1 Histórias de usuário 5

6.2 Regras de negócio 6

7 Requisitos não funcionais **7**

8 Limites desta etapa e próximos passos **7**

A Casos de teste de back-end e banco de dados **9**

B Casos de teste de interface **14**

1 Introdução

Este documento descreve os testes automatizados do sistema **Loja Virtual de Cupcakes**, os resultados obtidos, os defeitos encontrados e corrigidos e a rastreabilidade entre os testes e as histórias de usuário (US01 a US15) e regras de negócio (RN01 a RN15) definidas no Projeto Integrador I.

O objetivo é demonstrar, de forma repetível, que o comportamento esperado foi realizado. Os testes cobrem a **verificação** (o sistema foi construído conforme a especificação) e preparam a **validação** com usuários, etapa seguinte do projeto (seção 8).

Indicador	**Resultado**
Testes automatizados de back-end e banco (JUnit 5)	131 casos, todos aprovados
Testes automatizados de interface (Playwright)	31 casos, todos aprovados
Histórias de usuário com ao menos um teste	15 de 15
Regras de negócio com ao menos um teste	15 de 15
Defeitos encontrados e corrigidos	2 (seção 5)

Tabela 1: Resumo geral

2 Estratégia de testes

Os testes seguem uma pirâmide: muitos testes rápidos nas camadas internas e menos testes, mais amplos, nas camadas externas.

Nível	**O que verifica**	**Como**
Unitário	Funções isoladas: conversão JSON, hash de senha, sessões	JUnit 5, sem banco
Integração (DAO + banco)	Cada consulta e transação do Model contra o PostgreSQL real	JUnit 5; o banco é recriado antes de cada teste
Banco de dados	Restrições, índices únicos, triggers, views e permissões	JUnit 5 com SQL direto
Ponta a ponta (API)	Rotas HTTP, perfis, validações e fluxos completos (Controller + DAO + banco)	JUnit 5 sobe o servidor em uma porta livre e faz requisições HTTP
Interface (View)	Telas, mensagens e navegação no navegador	Playwright (Chromium), simulando o usuário

Tabela 2: Níveis de teste

Critérios adotados em cada caso: um caso de teste verifica um comportamento es-

perado e tem resultado objetivo (aprovado ou reprovado). Cada caso traz a indicação da história (USxx) e da regra de negócio (RNxx) a que se refere. Os casos incluem o caminho feliz, as entradas inválidas e os limites (por exemplo, quantidade 0, 1 e 100).

Isolamento: antes de cada teste de back-end, o esquema do banco de testes é apagado e recriado pelo mesmo script SQL do projeto, com os dados iniciais. Assim, nenhum teste depende do resultado de outro, e o próprio script de criação do banco é exercitado a cada execução.

3 Ambiente e como executar

3.1 Ambiente usado

- Java 21 (exige-se JDK 17 ou superior), JUnit 5 em modo console (`junit-platform-console-st`
  driver JDBC PostgreSQL 42.7.2.
- PostgreSQL 16, em um banco separado do banco de uso (`loja_cupcakes_test`).
- Interface: Python 3, Playwright e Chromium.

Proteção do banco

Os testes apagam e recriam o esquema `loja`. Por segurança, só rodam se o nome do banco contiver "test"; caso contrário, o teste aborta antes de tocar em qualquer dado.

3.2 Executar os testes de back-end

1. Crie o banco de testes (com o container do banco já ligado):

```
docker exec -it loja-db psql -U postgres -c "CREATE DATABASE loja_cupcakes_test"
```

2. Na pasta do projeto, rode (`testar.sh` no Linux/Mac):

```
testar.bat
```

O script compila o sistema e os testes e executa tudo. O relatório em XML fica na pasta `relatorios`. Se o banco usar outra senha ou porta, defina `TEST_DB_URL`, `TEST_DB_USER` e `TEST_DB_PASSWORD`.

3.3 Executar os testes de interface

1. Instale uma vez: `pip install playwright` e `playwright install chromium`.
2. Recrie o banco de testes com os dados iniciais e suba o sistema apontando para ele (`DB_URL` com o banco `loja_cupcakes_test`).

3. Rode: `python testes/interface/testar_interface.py http://localhost:8080`.

O script grava `relatorios/interface.json` e salva uma imagem da tela de cada teste reprovado.

4 Resultados

4.1 Back-end e banco de dados

Classe de teste	Nível	Foco	Casos	Aprovados
`JsonTest`	Unitário	Conversor JSON	10	10
`SenhaESessaoTest`	Unitário	Hash de senha e sessões	5	5
`UsuarioEEnderecoDaoTest`	Integração (DAO + banco)	Usuários, login e endereços	13	13
`CupcakeDaoTest`	Integração (DAO + banco)	Vitrine, filtro e produtos	15	15
`CarrinhoDaoTest`	Integração (DAO + banco)	Carrinho de compras	11	11
`PedidoDaoTest`	Integração (DAO + banco)	Pedido, pagamento, status e dashboard	29	29
`BancoDeDadosTest`	Banco de dados	Restrições, triggers, views e permissões	13	13
`ApiTest`	Ponta a ponta (API HTTP)	Rotas, perfis, validações e fluxos completos	35	35
Total			**131**	**131**

Tabela 3: Resumo dos testes automatizados de back-end e banco (JUnit 5)

Saída resumida do JUnit na última execução (todos os casos aprovados, tempo total de cerca de 25 segundos):

```
Test run finished after 24683 ms
[ 11 containers found      ]
[ 0 containers skipped     ]
[ 11 containers started    ]
[ 0 containers aborted     ]
[ 11 containers successful ]
[ 0 containers failed      ]
[ 131 tests found          ]
[ 0 tests skipped          ]
[ 131 tests started        ]
[ 0 tests aborted          ]
[ 131 tests successful     ]
[ 0 tests failed           ]
```

4.2 Interface

Os 31 casos de interface foram executados em um navegador Chromium automatizado, contra o sistema real e o banco de testes, e todos foram aprovados. Eles cobrem: tela inicial só com login, cadastro e login, vitrine e filtro, detalhes, carrinho na aba lateral, finalização do pedido, pagamento recusado e aprovado, Meus pedidos, painel do dono (dashboard, status, cadastro e edição de produto) e proteção contra HTML malicioso no nome de produto. A lista completa está no Apêndice B.

Para garantir que a verificação detecta erros, foi feito um controle negativo: uma checagem propositalmente falsa (esperar a loja visível antes do login) foi reprovada pelo mesmo mecanismo.

5 Defeitos encontrados e corrigidos

ID	Defeito	Correção	Teste de regressão
D1	Enviar um JSON cortado (por exemplo, `{"email":`) fazia o servidor responder erro 500 ("Erro interno") em vez de erro 400, porque o leitor lançava `StringIndexOutOfBoundsException`	`Json.lerObjeto` passou a converter o erro de leitura em `IllegalArgumentException`, que a API responde com 400	`JsonTest`: JSON malformado; `ApiTest`: JSON malformado devolve 400
D2	Enviar número decimal em campo inteiro (por exemplo, quantidade 1,5) gerava `ArithmeticException` e erro 500	`Json.longo` passou a lançar `IllegalArgumentException` com a mensagem "O campo ... deve ser um número inteiro"	`JsonTest`: decimal em inteiro; `ApiTest`: quantidade inválida (1,5)

Tabela 4: Defeitos encontrados nos testes

O D1 foi detectado pela execução dos testes (quatro casos reprovaram na primeira execução; dois deles eram erros na escrita dos próprios testes, que usavam um leitor de JSON que não lê listas, e foram ajustados). O D2 foi identificado na análise do mesmo trecho de código e recebeu testes de regressão antes da última execução completa. Após as correções, todos os casos foram aprovados.

6 Rastreabilidade

6.1 Histórias de usuário

A tabela mostra quantos casos de teste referenciam cada história. Um caso pode referenciar mais de uma.

ID	História	Back-end	Interface	Total	Resultado
US01	Visualizar vitrine	5	2	7	Aprovados
US02	Filtrar por categoria	6	1	7	Aprovados
US03	Ver detalhes	3	1	4	Aprovados
US04	Adicionar ao carrinho	9	1	10	Aprovados
US05	Remover do carrinho	3	2	5	Aprovados
US06	Visualizar carrinho	4	1	5	Aprovados
US07	Endereço de entrega	8	3	11	Aprovados
US08	Forma de pagamento	2	0	2	Aprovados
US09	Confirmar pedido	13	2	15	Aprovados
US10	Processar pagamento	8	1	9	Aprovados
US11	Acompanhar pedido	4	1	5	Aprovados
US12	Cadastrar produto	6	2	8	Aprovados
US13	Editar produto	5	1	6	Aprovados
US14	Atualizar status	8	2	10	Aprovados
US15	Dashboard de vendas	6	1	7	Aprovados

Histórias parcialmente implementadas

Alguns pontos do planejamento não estão implementados (por exemplo, seletor de quantidade no carrinho, consulta de CEP no ViaCEP, *webhook* do gateway e gráficos do dashboard). Os testes verificam o comportamento que existe hoje; esses pontos estão listados no capítulo de implementação do documento do projeto e entram nos testes quando forem implementados.

6.2 Regras de negócio

Regra	Casos	Verificada por (exemplos)
RN01	2	Produto inativo não aparece na vitrine da API; Vitrine lista somente produtos ativos
RN02	2	Categoria inválida gera erro 400; Categoria inexistente retorna lista vazia
RN03	2	Dono edita preço e estoque (detalhes mostram o dado novo); desativar tira da vitrine; Editar produto altera preço, estoque e categorias (dado atualizado)
RN04	2	Quantidade fora de 1 a 99 é recusada com 400; Quantidade zero é rejeitada e nada é gravado
RN05	1	Quantidade negativa é rejeitada
RN06	1	Carrinho calcula subtotal por item
RN07	4	Endereço de outro cliente ou inexistente é recusado; CEP inválido e UF em minúsculas são rejeitados pelo banco
RN08	2	Forma de pagamento inválida é rejeitada e nada é gravado; Forma de pagamento inválida devolve 400
RN09	5	Estoque insuficiente impede o pedido e nada é alterado; Sem carrinho aberto não é possível confirmar

Regra	Casos	Verificada por (exemplos)
RN10	3	Pagamento aprovado muda o pedido para PAGO; Pagamento processado precisa ter data de processamento
RN11	2	Status inexistente e pedido inexistente geram erro; Status inexistente, ausente ou pedido inexistente devolvem 400
RN12	3	Nome de produto repetido é rejeitado; Cadastro recusa campos obrigatórios vazios, preço zero e nome repetido
RN13	2	Editar produto inexistente gera erro; Editar produto inexistente devolve 400
RN14	6	Pedido entregue não pode mais mudar de status; Dono muda o status seguindo o fluxo e é barrado ao pular etapas
RN15	1	Vendas só contam pedidos já pagos e refletem o dado atual

7 Requisitos não funcionais

ID	Requisito	Situação nos testes
RNF01, RNF02	Vitrine em menos de 2 s e filtro em menos de 1 s	Verificado: ambos respondem em menos de 1 s (teste de desempenho da API)
RNF03	Interface responsiva	Não verificado automaticamente (a interface usa grade flexível; falta teste em celular)
RNF04	Persistência do carrinho	Verificado: o carrinho fica no banco (testes do carrinho e recarregamento da página)
RNF08, RNF12	Segurança	Verificado: perfis (401 e 403), hash de senha, permissões do banco, escape de HTML
RNF09, RNF13	Consistência e integridade	Verificado: transação do pedido, restrições e chaves do banco
RNF05, RNF06, RNF11, RNF14	Atualização instantânea e tempo real	Parcial: o carrinho atualiza na hora; o status dos pedidos usa consulta periódica
RNF10, RNF15	Disponibilidade e desempenho do dashboard	Não verificado (exigiria teste de carga)

Tabela 7: Requisitos não funcionais e verificação

8 Limites desta etapa e próximos passos

- **Validação com usuários:** os testes automatizados verificam a especificação. Falta a rodada de testes com cinco colegas ou profissionais, com formulário de bugs e sugestões, prevista na Situação 3 do projeto.

- **Não coberto:** teste de carga e concorrência (por exemplo, dois clientes comprando a última unidade ao mesmo tempo), compatibilidade com outros navegadores (a interface foi testada em Chromium), acessibilidade, uso em celular e testes de invasão.
- **Gateway de pagamento:** é simulado; o comportamento com um gateway real e com *webhook* ainda não foi testado.
- **Cancelamento:** cancelar um pedido não devolve as unidades ao estoque (limitação conhecida, documentada no manual de uso).
- **Cobertura de código:** não foi medida com ferramenta; a cobertura é demonstrada pela rastreabilidade entre testes, histórias e regras.

A Casos de teste de back-end e banco de dados

Legenda: **Refere-se a** indica a história (US), a regra de negócio (RN), o requisito não funcional (RNF) ou o tema (ACESSO, ESQUEMA, UTIL, ROBUSTEZ, VIEW) que o caso verifica.

JsonTest — Conversor JSON (10 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
1	UTIL	Escapa aspas, barra invertida e quebra de linha	OK
2	UTIL	Número decimal em campo inteiro gera IllegalArgumentException	OK
3	UTIL	Lê acentos e sequência \{}u	OK
4	UTIL	Campo numérico obrigatório ausente gera IllegalArgumentException	OK
5	UTIL	Corpo vazio vira objeto vazio	OK
6	UTIL	Converte lista de ids separada por vírgula	OK
7	UTIL	Serializa mapa com texto, número e booleano	OK
8	UTIL	JSON malformado gera IllegalArgumentException	OK
9	UTIL	Serializa nulo e lista de mapas	OK
10	UTIL	Lê objeto simples com tipos misturados	OK

SenhaESessaoTest — Hash de senha e sessões (5 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
11	UTIL	Sessão: tokens diferentes para logins diferentes	OK
12	UTIL	Sessão: cria, busca e encerra	OK
13	UTIL	Hash é determinístico, tem 64 caracteres e muda com a senha	OK
14	UTIL	Hash SHA-256 de '123456' confere com o valor conhecido	OK
15	UTIL	Sessão: token nulo ou desconhecido não autentica	OK

UsuarioEEnderecoDaoTest — Usuários, login e endereços (13 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
16	ACESSO	E-mail repetido é rejeitado e não deixa registro órfão	OK
17	US07	Lista os endereços do cliente	OK
18	US07	Endereço inativo não é listado	OK
19	ACESSO	Senha errada ou e-mail desconhecido não autentica	OK
20	US07	Salva novo endereço válido	OK
21	ACESSO	E-mail com formato inválido é rejeitado pelo banco	OK
22	ACESSO	Senha fica gravada apenas como hash	OK
23	ACESSO	Login de administrador devolve o perfil ADMINISTRADOR	OK
24	ACESSO	Usuário inativo não consegue entrar	OK

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
25	US07, RN07	CEP inválido e UF em minúsculas são rejeitados pelo banco	OK
26	ACESSO	Login de cliente com senha correta	OK
27	US07, RN07	Campos obrigatórios do endereço não podem faltar	OK
28	ACESSO	Cadastro cria usuário e cliente e permite login	OK

CupcakeDaoTest — Vitrine, filtro e produtos (15 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
29	US13	Produto inativado sai da vitrine mas continua na lista do dono	OK
30	US12	Cadastrar produto grava dados e categorias	OK
31	US03	Detalhes trazem descrição, ingredientes, preço e categorias	OK
32	US12, RN12	Nome de produto repetido é rejeitado	OK
33	US13, RN13	Editar produto inexistente gera erro	OK
34	US02	Filtro não mostra produto inativo da categoria	OK
35	US02	Lista de categorias vem ordenada por nome	OK
36	US01	Vitrine mostra os produtos em destaque primeiro	OK
37	US12	Falha ao gravar categoria desfaz o cadastro inteiro (transação)	OK
38	US03	Produto inexistente retorna nulo	OK
39	US02	Filtro por categoria retorna apenas produtos da categoria	OK
40	US02, RN02	Categoria inexistente retorna lista vazia	OK
41	US13, RN03	Editar produto altera preço, estoque e categorias (dado atualizado)	OK
42	US12, RN12	Preço zero e estoque negativo são rejeitados pelo banco	OK
43	US01, RN01	Vitrine lista somente produtos ativos	OK

CarrinhoDaoTest — Carrinho de compras (11 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
44	US05	Remover item que não está no carrinho não gera erro	OK
45	US04	Produto inexistente não pode ser adicionado	OK
46	US04	Adicionar item cria o carrinho aberto e grava a quantidade	OK
47	US04	Adicionar o mesmo produto de novo soma as quantidades	OK
48	US06	Carrinho de um cliente não aparece para outro	OK
49	US06, RN06	Carrinho calcula subtotal por item	OK
50	US06	Carrinho vazio retorna lista vazia	OK
51	US04, RN05	Quantidade negativa é rejeitada	OK
52	US04, RN04	Quantidade zero é rejeitada e nada é gravado	OK
53	US05	Remover item tira o produto e o total é recalculado	OK
54	US04	Produto inativo não pode ser adicionado	OK

PedidoDaoTest — Pedido, pagamento, status e dashboard (29 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
55	US14, RN11	Status inexistente e pedido inexistente geram erro	OK
56	US10	Cliente não consegue pagar o pedido de outro cliente	OK
57	US10	Pagamento recusado mantém o pedido aguardando e permite nova tentativa	OK
58	US11	Pedido nasce com o status inicial registrado no histórico	OK
59	US09	Endereço de entrega é copiado para o pedido	OK
60	US09, RN09	Estoque insuficiente impede o pedido e nada é alterado	OK
61	US09, RN09	Sem carrinho aberto não é possível confirmar	OK
62	US10, RN10	Pagamento aprovado muda o pedido para PAGO	OK
63	US15, RN15	Vendas só contam pedidos já pagos e refletem o dado atual	OK
64	US08, RN08	Forma de pagamento inválida é rejeitada e nada é gravado	OK
65	US09	Confirmar cria pedido aguardando pagamento com total = itens + taxa	OK
66	US09, RN07	Endereço de outro cliente ou inexistente é recusado	OK
67	US11	Cliente vê apenas os seus pedidos, com status e pagamento	OK
68	US09	Depois de confirmar, o cliente pode montar um novo carrinho	OK
69	US14, RN14	Pedido entregue não pode mais mudar de status	OK
70	US10	Não é possível pagar um pedido que já foi pago	OK
71	US09, RN09	Carrinho vazio não gera pedido	OK
72	US14	Histórico guarda quem alterou o status	OK
73	US09	Preço do item fica gravado: mudar o preço depois não altera o pedido	OK
74	US09, RN09	Produto inativado depois de entrar no carrinho impede o pedido	OK
75	US11	Pedidos aparecem do mais novo para o mais antigo	OK
76	US09	Confirmar fecha o carrinho e esvazia a lista	OK
77	US14, RN14	Pedido pode ser cancelado enquanto aguarda pagamento	OK
78	US09	Confirmar baixa o estoque dos produtos	OK
79	US15	Pedido cancelado não entra nas vendas	OK
80	US14, RN14	Pular etapa do fluxo é rejeitado	OK
81	US15	Mais vendidos vem ordenado por unidades	OK
82	US14, RN14	Fluxo completo de status: PAGO, EM_PREPARO, ENVIADO e ENTREGUE	OK
83	US15	Relatório de estoque classifica BAIXO, OK e ESGOTADO	OK

BancoDeDadosTest — Restrições, triggers, views e permissões (13 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
84	US04, ESQUEMA	Um cliente só pode ter um carrinho aberto	OK
85	RNF08	Papel da aplicação não pode apagar pedidos nem pagamentos	OK
86	ACESSO	Senhas de demonstração estão guardadas como hash	OK
87	US01, US15	Views de vitrine e estoque respondem com os dados iniciais	OK
88	ESQUEMA	Subtotal do item do pedido é calculado pelo banco	OK

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
89	ESQUEMA	Valores monetários negativos são rejeitados	OK
90	US10, RN10	Pagamento processado precisa ter data de processamento	OK
91	ESQUEMA	Nome de categoria é único	OK
92	ESQUEMA	Produto com pedidos não pode ser apagado	OK
93	RN14	Tabela de transições tem as 7 regras do fluxo	OK
94	US10, ESQUEMA	Um pedido não pode ter dois pagamentos aprovados	OK
95	US07, ESQUEMA	Só um endereço principal ativo por cliente	OK
96	ESQUEMA	Administrador não pode ser cadastrado como cliente	OK

ApiTest — Rotas, perfis, validações e fluxos completos (35 casos)

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
97	ROBUSTEZ	Método HTTP não permitido devolve 405; rota inexistente devolve 404	OK
98	US03	Detalhes do produto: 200 se existe e ativo, 404 se não	OK
99	ACESSO	Rotas protegidas exigem login (401) e rejeitam token inventado	OK
100	ACESSO	Login válido devolve token, nome e perfil	OK
101	US12	Dono cadastra produto novo e ele aparece na vitrine	OK
102	ACESSO	Cadastro rejeita e-mail repetido, senha curta e telefone inválido	OK
103	US13, RN03	Dono edita preço e estoque (detalhes mostram o dado novo); desativar tira da vitrine	OK
104	US04, RN04	Quantidade fora de 1 a 99 é recusada com 400	OK
105	US14, RN11	Status inexistente, ausente ou pedido inexistente devolvem 400	OK
106	ACESSO	Cadastro de cliente novo e login em seguida	OK
107	US09, RN09	Confirmar com carrinho vazio ou sem endereço devolve 400	OK
108	US02, RN02	Categoria inválida gera erro 400	OK
109	ROBUSTEZ	JSON malformado devolve 400 (e não erro 500)	OK
110	US01	Vitrine é pública e lista os 5 cupcakes ativos	OK
111	US11	Cliente só vê os próprios pedidos	OK
112	US10, RN10	Pagamento recusado e depois aprovado leva o pedido a PAGO	OK
113	US01, RN01	Produto inativo não aparece na vitrine da API	OK
114	ACESSO	Logout invalida o token	OK
115	ACESSO	Cliente não acessa o painel do dono (403) e o dono não usa o carrinho (403)	OK
116	US07	Cadastrar endereço válido e listá-lo	OK
117	VIEW	Páginas estáticas são servidas; fora da pasta public dá 404	OK
118	ACESSO	Login inválido devolve 401; campos ausentes devolvem 400	OK
119	US12, RN12	Cadastro recusa campos obrigatórios vazios, preço zero e nome repetido	OK
120	US14, RN14	Dono muda o status seguindo o fluxo e é barrado ao pular etapas	OK
121	RNF01, RNF02	Vitrine e filtro respondem em menos de 1 segundo	OK
122	US13, RN13	Editar produto inexistente devolve 400	OK

N.º	Refere-se a	Caso de teste (comportamento esperado)	Result.
123	ROBUSTEZ	Texto com aspas e acentos no nome do produto volta íntegro no JSON	OK
124	US09	Confirmar pedido: 201, estoque baixa e carrinho esvazia	OK
125	US08, RN08	Forma de pagamento inválida devolve 400	OK
126	US10	Pedido já pago não pode ser pago de novo; pedido alheio também não	OK
127	US07, RN07	Endereço com CEP, UF ou campos obrigatórios inválidos é recusado	OK
128	US15	Dashboard devolve vendas, mais vendidos e estoque	OK
129	US02	Filtro por categoria retorna só os produtos dela	OK
130	US04, US06	Adicionar ao carrinho devolve itens e total atualizados	OK
131	US05	Remover item do carrinho atualiza o total	OK

B Casos de teste de interface

Cód.	Refere-se a	Caso de teste de interface (comportamento esperado)	Result.
UI01	ACESSO	Ao abrir, aparece somente a tela de login (loja e vitrine ocultas)	OK
UI02	ACESSO	Login com senha errada mostra mensagem de erro e continua no login	OK
UI03	ACESSO	Cadastro de conta nova volta ao login com o e-mail preenchido	OK
UI04	ACESSO	Cadastro com senha curta mostra erro	OK
UI05	US01	Após o login aparece a vitrine com 5 cupcakes e some a tela de login	OK
UI06	US01	Produtos em destaque aparecem primeiro	OK
UI07	US02	Filtro 'Vegano' mostra 1 cupcake e 'Todas' volta aos 5	OK
UI08	US03	Detalhes abre janela com ingredientes e preço	OK
UI09	US04	Adicionar mostra aviso e atualiza o contador do carrinho	OK
UI10	US06	Aba lateral abre o carrinho com 2 itens e total R$ 31,50	OK
UI11	US05	Remover item atualiza lista, total e contador	OK
UI12	US05	Carrinho vazio mostra mensagem e desabilita Finalizar pedido	OK
UI13	ACESSO	Clicar fora da aba lateral (ou no X) fecha o painel	OK
UI14	US09	Finalizar pedido: escolher endereço da Maria e confirmar abre o pagamento	OK
UI15	US10	Pagamento recusado avisa o cliente; nova tentativa é aprovada	OK
UI16	US11	Meus pedidos mostra o pedido com status PAGO e pagamento APROVADO	OK
UI17	ACESSO	Recarregar a página mantém o usuário logado	OK
UI18	ACESSO	Cliente logado não vê o painel do dono em admin.html	OK
UI19	ACESSO	Sair devolve à tela de login	OK
UI20	US07	Sem endereço, a janela de finalização já abre o formulário de novo endereço	OK
UI21	US07	CEP inválido mostra erro e o endereço não é salvo	OK
UI22	US07	Endereço válido é salvo e já fica selecionado	OK
UI23	US09	Pedido confirmado fica 'aguardando pagamento' com botão Pagar agora	OK
UI24	ACESSO	Administrador que entra pela tela de login vai direto ao painel do dono	OK
UI25	US15	Dashboard mostra vendas, mais vendidos e estoque	OK
UI26	US14	Dono muda o status de PAGO para EM_PREPARO	OK
UI27	US14	Pular etapa (EM_PREPARO para ENTREGUE) mostra a mensagem de erro	OK
UI28	US12	Dono cadastra produto novo e ele aparece na lista de produtos	OK
UI29	US12	Cadastro com preço vazio é barrado na tela	OK
UI30	US13	Dono edita o estoque de um produto (repor estoque)	OK
UI31	SEGURANÇA	Nome de produto com HTML aparece como texto (sem executar código)	OK